

Document Number: P0696R0
Date: 2017-06-19
Audience: Evolution Working Group
Reply-to: Tom Honermann <tom@honermann.net>

Remove abbreviated functions and template-introduction syntax from the Concepts TS

- [Introduction](#)
- [Motivation](#)
- [Wording](#)
- [Acknowledgements](#)
- [References](#)

Introduction

The abbreviated function and template introducer syntax features defined in the Concepts TS [\[Concepts\]](#) have proven to be controversial as evidenced by discussion within the committee reflectors [\[ExploringConcepts\]](#) and the [P0587R0](#) [\[P0587R0\]](#) and [P0464R2](#) [\[P0464R2\]](#) paper submissions. This paper proposes removing these features from the Concepts TS with the goal of increasing consensus on adopting the remaining Concepts TS functionality into the current working paper.

Removal of support for abbreviated functions means that the following example that is currently well-defined by the Concepts TS will no longer be valid:

```
template<typename> concept bool C = true;

void af1(auto);      // Previously ok; declares an abbreviated function (template)
                    // with a single function parameter with an unconstrained type.
                    // Now ill-formed.
void af2(C);         // Previously ok; declares an abbreviated function (template)
                    // with a single function parameter with a constrained type.
                    // Now ill-formed.
```

This paper does **not** propose removing support for declaring functions and function templates with return types containing placeholders. The following examples retain their current behavior under the Concepts TS:

```
template<typename, typename> class ct {};

ct<auto,auto> f1();   // Ok; declares a function with a return type
                    // that requires the template arguments of ct
                    // to be deduced from a return statement.
ct<C,C> f2();        // Ok; declares a function with a return type
                    // that requires the template arguments of ct
                    // to be deduced from a return statement, to be deduced
                    // to the same type, and to satisfy C.
```

Removal of support for template introducers means that the following examples that are currently well-defined by the Concepts TS will no longer be valid:

```
C{T} void ft1(T);    // Previously ok; declares a function template
                    // with a single constrained type template parameter.
                    // Now ill-formed.
C{T} class ct1;      // Previously ok; declares a class template
                    // with a single constrained type template parameter.
                    // Now ill-formed.
```

This paper does **not** propose removing support for declaring parameters of generic lambdas with *constrained-type-specifiers*, nor does it alter the current requirement that the same types be deduced for equivalently constrained parameters. The following examples retain their current behavior under the Concepts TS:

```
auto gl1 = [](C,C) { return 1; };
auto v1 = gl1(0,0);    // Ok
auto v2 = gl1(0,'c'); // Ill-formed; different types deduced for C
```

Motivation

The abbreviated function and template introducer syntax defined by the Concepts TS is intended to reduce boilerplate, improve readability, and ultimately, make generic programming more accessible as elaborated in [P0557R0](#) [\[P0557R0\]](#). These are laudible goals and ones the author is in favor of continued pursuit. However, the features currently defined in the Concepts TS raise concerns regarding mutation of code undergoing maintenance and difficulties programmers may face in navigating the differences in requirements and behaviors exhibited by functions and function templates that are made more subtle by the current design.

Consider the following function declaration. Does it declare a function or a function template?

```
void f(X x) {}
```

Prior to the Concepts TS, this unambiguously declares a non-template function. However, if *x* names a concept, then *f* is an abbreviated function equivalent to the following function template:

```
template<X T> void f(T x) {}
```

This is exactly the intent as described in [P0557R0](#) [\[P0557R0\]](#); one may now write generic functions as familiar to any C programmer without the boilerplate currently required to author a function template!

The problem is, this simplicity is an illusion that dissolves under further inspection as illustrated in the following examples.

```
void f1(X&& x) {}
```

If *x* names a concept, then parameter *x* is a forwarding reference; otherwise, it is an rvalue reference and *f1* may only be called with an rvalue argument.

```
int f2(X x) {
    static int invocation_count;
    return ++invocation_count;
}
```

f2 maintains a count of either every invocation of *f2*, or every invocation of an instantiated specialization of *f2* depending on whether *x* names a concept or a type.

```
void f3(X x) {}
```

f3 may be moved from a primary source file to a header if *x* names a concept, but doing so risks introducing multiple definition errors if *x* names a type.

```
void f4(int i) {}
```

If *f4* were modified to:

```
void f4(int i, X x) {}
```

then its definition will change from a non-template function to a template function if `x` names a concept, but will remain a function if `x` names a type.

```
void f5(Ziggle z, Piddle p) {
    Ziggle::type x1;           // Ok if Ziggle is a type, ill-formed if it's a concept.
    decltype(p)::type x2;      // Ok if Piddle is a type, needs typename if it is a concept.
    typename decltype(z)::type x3; // Ok if Ziggle is a concept, ill-formed if it is a type.
}
```

`f5` illustrates the increased subtlety in determining when `typename` disambiguation is required. The presence of a *template-parameter-list* enables such determination without having to be familiar with the particular identifiers:

```
template<Ziggle Z>
void f5(Z z, Piddle p) {
    Ziggle::type x1;           // Ill-formed; Ziggle is a concept.
    decltype(p)::type x2;      // Ok; decltype(p) is not dependent.
    typename decltype(z)::type x3; // Ok; decltype(z) is dependent.
}
```

Abbreviated functions have the behavior that placeholders specified within the declared return type of the function that match a *constrained-type-specifier* used in a parameter type are not deduced, but are rather replaced by the deduced type for the matching parameter(s). However, this behavior doesn't hold if the same return type is specified as a *trailing-return-type*: [*Note*: It is not clear that this is the intent of the Concepts TS, but this is the behavior exhibited by gcc. — *end note*]

```
template<typename> concept bool C = true;
template<typename> class ct {};

ct<C> f6(C) { return ct<int>{}; }
auto f7(C) -> ct<C> { return ct<int>{}; }

f6('c'); // Ill-formed; different types deduced for each C
          // and ct<int> is not convertible to ct<char>.
f7('c'); // Ok; C in the parameter list deduced as char
          // from the function call argument; C in the trailing
          // return type deduced as int from the return statement.
```

A semantic difference based on whether the return type of a function is specified in the declared return type as opposed to a *trailing-return-type* is novel and introduces the possibility of subtle defects being introduced due to a change a programmer expects to be purely stylistic. Similarly, this behavior introduces the possibility that a change to the declaration of a function parameter may alter the meaning of the return type: [*Note*: gcc 6.2-7.1 rejects both of the following cases as it (presumably erroneously) replaces `C` in the return type with the invented template parameter for the first parameter regardless of whether the first parameter is declared with a matching *constrained-type-specifier*. — *end note*]

```
ct<C> f8(auto) { return ct<int>{}; }
ct<C> f9(C) { return ct<int>{}; }

f8('c'); // Ok; C deduced as int from the return statement.
f9('c'); // Ill-formed; C deduced as char from the function
          // argument and ct<int> is not convertible to ct<char>.
```

The template introduction syntax does not suffer concerns like those above. The primary concern with it is aesthetic; that it lacks cohesiveness with the rest of the language as described in [P0587R0](#) [P0587R0]. The choice to remove it as proposed in this paper is based on a perception that an alternate syntax to replace abbreviated function syntax might also encompass the goals of template introductions.

Few of the concerns raised above are applicable to every template declaration and it has not been claimed that either of the terse syntaxes is the best fit for all template declarations. Is it not therefore reasonable for the use of these syntaxes to be a matter of

code style? Do tools such as IDEs that provide semantic markup not help to counteract these concerns?

The examples above suggest that guidelines for the usage of abbreviated function syntax can be crafted. A simple guideline might look like:

Write a function template instead of an abbreviated function when:

- parameters declared with && are present.
- the parameter list consists of non-dependent user declared types and constrained types.
- dependent member type access is needed.
- static local variables are present.

Write abbreviated functions that have return types containing placeholders using a *trailing-return-type*.

Whether the concerns presented in this paper rise above a level adequately addressed by guidelines and coding standards is a matter of judgement and no arguments are presented here to attempt to establish a litmus test.

IDEs and other tools that provide semantic markup are certainly helpful. However, not all programmers use IDEs and most of us use tools that lack semantic awareness; particularly for tools used for code review, diffing, and merging. Relying on semantic awareness to help elucidate code is likely to result in disappointment in various parts of our collective work flows for the foreseeable future.

Wording

☐ Hide deleted text

These changes are relative to a draft of the Concepts TS working paper being prepared for the 2017 Toronto pre-meeting mailing [\[ConceptsWP\]](#)

Change the added paragraph 6 inserted in 8.1.4.2 [expr.prim.id.qual]:

In a *nested-name-specifier* of the form `auto::` or `C::`, where `C` is a *constrained-type-name*, that *nested-name-specifier* designates a placeholder that will be replaced later according to the rules for placeholder deduction in 10.1.7.4. If a placeholder designated by a *constrained-type-specifier* is not a placeholder type, the program is ill-formed. [*Note*: A *constrained-type-specifier* can designate a placeholder for a non-type or template (10.1.7.4.2). — *end note*] The replacement type deduced for a placeholder shall be a class or enumeration type. [*Example*:

```
template<typename T> concept bool C = sizeof(T) == sizeof(int);
template<int N> concept bool D = true;

struct S1 { int n; };
struct S2 { char c; };
struct S3 { struct X { using Y = int; }; };

int auto::* p1 = &S1::n; // auto deduced as S1
int D::* p2 = &S1::n;    // error: D does not designate a placeholder type
int C::* p3 = &S1::n;    // OK: C deduced as S1
char C::* p4 = &S2::c;    // error: deduction fails because constraints are not satisfied

void f(typename auto::X::Y);
f(S1()); // error: auto cannot be deduced from S1()
f<S3>(0); // OK
```

~~In the declaration of `f`, the placeholder appears in a non-deduced context (17.8.2.5). It may be replaced later through the explicit specification of template arguments.~~ — *end example*]

Change in 8.1.5.1 [expr.prim.lambda.closure]:

Modify paragraph 3 so that the meaning of a generic lambda is defined in terms of its ~~abbreviated~~ member function template call operator.

[...] The closure type for a generic lambda has a public inline function call operator member template that is an abbreviated function template whose parameters and return type are derived from the lambda-expression's parameter-declaration-clause and trailing-return-type according to the rules in (11.3.5)(10.1.7.4.1).

Add the following example after those in paragraph 3 in the C++ Standard.

[Example:

```
template<typename T> concept bool C = true;

auto gl = [](C& a, C* b) { a = *b; }; // OK: denotes a generic lambda

struct Fun {
    template<C T> auto operator()(CT& a, CT* b) const { a = *b; }
} fun;
```

C is a constrained-type-specifier, signifying that the lambda is generic. The generic lambda gl and the function object fun have equivalent behavior when called with the same arguments.
— end example]

Change in 8.1.7.3 [expr.prim.req.compound] paragraph 1 near the end of the example:

[...]

```
template<typename T> concept bool C() { return true; }

template<typename T> concept bool C5 =
    requires(T x) {
        {f(x)} -> const C&;
    };
```

The compound-requirement in C5 introduces two constraints: an expression constraint for $f(x)$, and a deduction constraint requiring that overload resolution succeeds for the call $g(f(x))$ where g is the following invented abbreviated function template.

```
template<C T> void g(const CT&);
```

— end example]

Change within the replacement text in 10.1.7.4 [dcl.spec.auto] paragraph 1:

Replace paragraph 1 with the text below.

[...] The type-specifiers `auto` and `decltype(auto)` and constrained-type-specifiers designate a placeholder (type, non-type, or template) that will be replaced later, either through deduction or an explicit specification. The `auto` and `decltype(auto)` type-specifiers designate placeholder types; a constrained-type-specifier can also designate placeholders for values and templates. [Note: The deduction of placeholders is done through the invention of template parameters as described in 10.1.7.4.1 and 11.3.5. — end note] Placeholders are also used to signify that a lambda is a generic lambda (8.1.5), that a function declaration is an abbreviated function template (11.3.5), or that a trailing-return-type in a compound-requirement (8.1.7.3) introduces an argument deduction constraint (17.10.1.6). The `auto` type-specifier is also used to introduce a function type having a trailing-return-type or to introduce a structured binding declaration 11.5. [Note: A nested-name-specifier can also include placeholders (8.1). Replacements for those placeholders are determined according to the rules in this section. — end note]

Remove the allowance for placeholders appearing in the parameter type of a function declaration in the added text in 10.1.7.4 [dcl.spec.auto] paragraph 3:

[...]

— end example] Similarly, if a placeholder appears in a parameter type of a function declaration, the function declaration declares an abbreviated function template (11.3.5). [Example:

```
void f(const auto&, int); // OK: an abbreviated function template
```

— end example]

Remove the added paragraph 4 inserted in 10.1.7.4 [dcl.spec.auto]:

Add the following after paragraph 3 to describe when constrained-type-specifiers in the return type refer to template parameters:

A constrained-type-specifier ϵ_1 within the declared return type of an abbreviated function template declaration does not designate a placeholder if its introduced constraint-expression (10.1.7.4.2) is determined to be equivalent, using the rules in 17.6.5.1 for comparing expressions, to the introduced constraint-expression for a constrained-type-specifier ϵ_2 in the parameter-declaration-clause of that function declaration. Instead, ϵ_1 is replaced by the template parameter invented for ϵ_2 (11.3.5). [Example:

```
template<typename T> concept bool C = true;

template<typename... T> struct Tuple;

C const& f1(C); // has one template parameter and no deduced return type
Tuple<C...> f2(C); // has one template parameter and a deduced return type
```

In the declaration f_1 , the constraint-expression introduced by the constrained-type-specifiers in the parameter-declaration-clause and return type are equivalent; they would both introduce the expression $C \leq T$, for some invented template parameter T . In f_2 , the use of C in the return type would introduce the constraint-expression $(C \leq T \ \&\& \dots)$, which is distinct from the constraint-expression $C \leq T$ introduced by the invented constrained parameter (17.1) for the constrained-type-specifier in the parameter-declaration-clause according to the rules in 11.3.5.

— end example

Renumber the added paragraphs 5 and 6 added to 10.1.7.4 [dcl.spec.auto] to 4 and 5 respectively.

Replace the changes to 10.1.7.4.1 [dcl.spec.auto.deduct] paragraph 2 with the following:

A type T containing a placeholder type **placeholders**, and a corresponding initializer e , are determined as follows:

(2.1) — for a non-discarded return statement that occurs in a function declared with a return type that contains a placeholder type **placeholders**, T is the declared return type and e is the operand of the return statement. If the return statement has no operand, then e is `void()`;

(2.2) — for a variable declared with a type that contains a placeholder type **placeholders**, T is the declared type of the variable and e is the initializer. If the initialization is direct-list-initialization, the initializer shall be a braced-init-list containing only a single assignment-expression and e is the assignment-expression;

(2.3) — for a non-type template parameter declared with a type that contains a placeholder type **placeholders**, T is the declared type of the non-type template parameter and e is the corresponding template argument;

(2.4) — for a parameter of a generic lambda-expression declared with a type that contains placeholders, τ is the declared type of the parameter and e is the corresponding argument in an invocation of the generic lambda's function call operator template;

(2.5) — for an argument deduction constraint (17.10.1.6), τ is the trailing-return-type, and e is the expression of the corresponding compound-requirement.

In the case of a return statement with no operand or with an operand of type void, τ shall be either `decltype(auto)` ~~or~~, `cv auto`, or a constrained-type-specifier.

Correct the changes to 10.1.7.4.1 [dcl.spec.auto.deduct] paragraph 3 to reference paragraph 4 from the C++ standard.

Replace the changes to 10.1.7.4.1 [dcl.spec.auto.deduct] paragraph 4 with the following:

If the placeholder is placeholders include the auto type-specifier or a constrained-type-specifier, the deduced type τ' replacing τ is determined using the rules for template argument deduction. If τ corresponds to the type of a parameter of a generic lambda-expression, then template argument deduction is performed for all parameter types containing placeholders together. ~~Obtain P from τ by replacing the occurrences of `auto` with either a new invented type template parameter U or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`.~~ Obtain a type P from τ by replacing each placeholder as follows:

(4.1) — if the initialization is a copy-list-initialization and a placeholder is a decl-specifier of the decl-specifier-seq of the variable declaration, replace that occurrence of the placeholder with `std::initializer_list<U>` where U is an invented type template parameter;

(4.2) — otherwise, if the placeholder is designated by the auto type-specifier, replace the occurrence with a new invented type template-parameter or, if τ corresponds to a function parameter pack, with a new invented type template parameter pack;

(4.3) — otherwise, the placeholder is designated by a constrained-type-specifier. If a placeholder in τ or, for parameters of generic lambda-expressions, any preceding parameter, has already been replaced by an invented constrained-parameter (17.1) with constraint-expressions (17.1) equivalent according to the rules for comparing expressions in 17.6.5.1 to a constrained-parameter whose qualified-concept-name is that of the constrained-type-specifier, then replace the occurrence with the existing invented constrained-parameter. Otherwise, replace the occurrence with a new invented constrained-parameter whose qualified-concept-name is that of the constrained-type-specifier.

Deduce a value for each invented template parameter using the rules of template argument deduction from a function call (17.8.2.1), where P is a function template parameter type and the corresponding argument is e . [Note: multiple function template parameters will be present when deducing types for generic lambda-expressions with multiple parameters with types containing placeholders. — end note] If the deduction fails, the declaration is ill-formed. If any placeholders in the declared type were introduced by a constrained-type-specifier, then define c to be a constraint-expression as follows:

(4.4) — if there is a single constrained-type-specifier, then c is the constraint-expression introduced by the invented template constrained-parameter (17.1) corresponding to that constrained-type-specifier;

(4.5) — otherwise, c is the logical-and-expression (8.14) whose operands are the constraint-expressions introduced by the invented template constrained-parameters corresponding to each constrained-type-specifier, in order of appearance.

If the normalized constraint for c (17.10.2) is not satisfied by the deduced values, the declaration is ill-formed. Otherwise, τ' is obtained by substituting the deduced u values for the invented type template parameters into P .

[Example:

```
auto x1 = { 1, 2 }; // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 }; // error: cannot deduce element type
```



```

auto x3{ 1, 2 };           // error: not a single element
auto x4 = { 3 };           // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };              // decltype(x5) is int

template<typename T> struct Vec { };
template<typename T> Vec<T> make_vec(std::initializer_list<T>) { return Vec<T>{}; }

template<typename... Ts> struct Tuple { };
template<typename... Ts> auto make_tup(Ts... args) { return Tuple<Ts...>{}; }

auto& x3 = *x1.begin();     // OK: decltype(x3) is int&
const auto* p = &x3;        // OK: decltype(p) is const int*
Vec<auto> v1 = make_vec({1, 2, 3}); // OK: decltype(v1) is Vec<int>
Vec<auto> v2 = {1, 2, 3};    // error: type deduction fails
Tuple<auto...> v3 = make_tup(0, 'a'); // OK: decltype(v3) is Tuple<int, char>

```

— end example]

[Example:

```
const auto &i = expr;
```

The type of *i* is the deduced type of the parameter *u* in the call *f(expr)* of the following invented function template:

```
template <class U> void f(const U& u);
```

— end example]

[Example:

```

template<typename F, typename S> struct Pair;
template<typename T, typename U> Pair<T, U> make_pair(T, U);

struct S { void mfn(bool); } s;
int fn(char, double);

Pair<auto (*) (auto, auto), auto (auto::*)(auto)> p = make_pair(fn, &S::mfn);

```

The declared type of *p* is the deduced type of the parameter *x* in the call of *g(make_pair(fn, &S::mfn))* of the following invented function template:

```

template<class T1, class T2, class T3, class T4, class T5, class T6>
void g(Pair< T1(*) (T2, T3), T4 (T5::*)(T6)> x);

```

— end example]

[Example:

```

template<typename T> concept bool C = true;

const C* cv = expr;

```

The type of *cv* is deduced from the parameter *p1* in the call *f1(expr)* of the following invented function:

```
template<C T> void f1(const T* p1);
```

— end example]

[Example:

```
auto cf(int) -> Pair<C, C> { return expr; }
```


The return type of *cf* is deduced from the parameter *p2* in the call *f2(expr)* of the following invented function:

```
template<C T> void f2(Pair<T, T>);
```

Both constrained-type-specifiers in the return type of *cf* correspond to the same invented template parameter.
— end example]

[Example:

```
namespace N {  
    template<typename T> concept bool C = true;  
}  
template<typename T> concept bool C = true;  
template<typename T, int> concept bool D = true;  
template<typename, int = 0> concept bool E = true;  
  
auto gl1 = [] (C, D<0>) {};
```

The constrained-type-specifiers *C* and *D<0>* correspond to distinct invented template parameters in the declaration of the function call operator *template* of the closure type.

```
auto gl2 = [] (C a, C b) {};
```

The types of *a* and *b* are the same invented template type parameter.

```
auto gl3 = [] (C& a, C* b) {};
```

The type of *a* is a reference to an invented template type parameter *T*, and the type of *b* is a pointer to *T*.

```
auto gl4 = [] (N::C a, C b) {};  
auto gl5 = [] (D<0> a, D<1> b) {};
```

In both lambda-expressions, the parameters *a* and *b* have different invented template type parameters.

```
auto gl6 = [] (E a, E<> b, E<0> c) {};
```

The types of *a*, *b*, and *c* are the same because the constrained-type-specifiers *E*, *E<>*, and *E<0>* all associate the constraint-expression *E<T, 0>*, where *T* is an invented template type parameter.

```
auto gl7 = [] (C head, C... tail) {};
```

The types of *head* and *tail* are different. Their respective introduced constraint-expressions are *C<T>* and *(C<U> && ...)*, where *T* is the template parameter invented for *head* and *U* is the template parameter invented for *tail* (17.1).

— end example]

Change the example in 10.1.7.4.2 [dcl.spec.auto.constr] paragraph 1:

[...]

[Example:

```
template<typename T> concept bool C1 = false;  
template<int N> concept bool C2 = false;  
template<template<typename> class X> concept bool C3 = false;
```

```

template<typename T, int N> class Array { };
template<typename T, template<typename> class A> class Stack { };
template<typename T> class Alloc { };

void f1(C1); // C1 designates a placeholder type
void f2(Array<auto, C2>); // C2 designates a placeholder for an integer value
void f3(Stack<auto, C3>); // C3 designates a placeholder for a class template
C1 f1(); // C1 designates a placeholder type
Array<auto, C2> f2(); // C2 designates a placeholder for an integer value
Stack<auto, C3> f3(); // C3 designates a placeholder for a class template

```

– end example

Change in 10.1.7.4.2 [dcl.spec.auto.constr] paragraph 2:

An identifier is a concept-name if it refers to a set of concept definitions (10.1.8). [Note: The set of concepts has multiple members only when referring to a set of overloaded function concepts. There is at most one member of this set when a concept-name refers to a variable concept. – end note] [Example:

```

template<typename T> concept bool C() { return true; } // #1
template<typename T, typename U> concept bool C() { return true; } // #2
template<typename T> concept bool D = true; // #3

template<C T> void f(T); // OK: the set of concepts referred to by C includes both #1 and #2;
                        // concept resolution (17.10.4) selects #1.
template<D T> void g(T); // OK: the concept-name D refers only to #3

```

– end example]

Change in 10.1.7.4.2 [dcl.spec.auto.constr] paragraph 3:

A partial-concept-id is a concept-name followed by a sequence of template-arguments. [Example:

```

template<typename T, int N = 0> concept bool Seq = true;

template<Seq<3> T> void f1(Seq<3> T); // OK
template<Seq<> T> void f2(Seq<> T); // OK

```

– end example]

Change in 10.1.7.4.2 [dcl.spec.auto.constr] paragraph 5:

[...] The rules for inventing template parameters corresponding to placeholders in the parameter-declaration-clause of a lambda-expression (8.1.2) or function declaration (11.3.5) are described in 11.3.5 10.1.7.4.1 [...]

Remove the added paragraphs 18-21 inserted in 11.3.5 [dcl.fct]:

~~Add the following paragraphs after paragraph 17.~~

~~18 An abbreviated function template is a function declaration [...] – end note]~~

~~19 Each template parameter is invented as follows.~~

~~[...]~~

~~where T is the template parameter invented for head and U is the template parameter invented for tail (14.1).~~

~~20 The adjusted function parameters of an abbreviated function template are derived [...] – end example]~~

~~[...]~~

~~19 A function template can be an abbreviated function template. [...]~~

~~[...]~~

~~– end example]~~

Change in 17 [temp]:

Modify the template-declaration grammar in paragraph 1 to allow a template declaration ~~introduced by a concept~~ to specify a **requires-clause**.

Change in 17 [temp] paragraph 1:

```
template-declaration:
    template < template-parameter-list > requires-clauseopt declaration
template introduction declaration
requires-clause:
    requires constraint-expression
```

Change in the added 17 [temp] paragraph 7:

A template-declaration is written in terms of its template parameters. These parameters are declared explicitly in a template-parameter-list (14.1), ~~or they are introduced by a template introduction (14.2)~~. The optional requires-clause following a template-parameter-list allows the specification of constraints (14.10.2) on template arguments (14.4).

Remove the added 17.2 [temp.intro]:

~~Add this section after 17.1.~~

~~1 A template introduction [...]~~

~~2 The concept designated by [...]~~

~~3 A concept referred to [...]~~

~~4 An introduced template parameter [...]~~

~~5 A template introduction introduces [...]~~

~~6 A template declared by a template introduction [...]~~

Change in 17.10.1.6 [temp.constr.deduct] paragraph 2:

To determine if an argument deduction constraint is satisfied, invent an ~~abbreviated~~ function template f with one parameter whose type is T ~~(11.3.5)~~ **(10.1.7.4.1)**. The constraint is satisfied if the resolution of the function call $f(E)$ succeeds (16.3). [Note: Overload resolution succeeds when values are deduced for all invented template parameters in f that correspond to the placeholders in T , and the constraints associated by any constrained-type-specifiers are satisfied. – end note] [Example:

```
template<typename T, typename U> struct Pair;

template<typename T>
    concept bool C1() { return true; }

template<typename T>
    concept bool C2() { return requires(T t) { { *t } -> Pair<C1&, auto>; }; }

template<C2 T> void g(T);

g((int*)nullptr); // error: constraints not satisfied.
```

The invented ~~abbreviated~~ function template f for the compound-requirement in $C2$ is:

```
template<C1 T1, typename T2> void f(Pair<C1T1&, autoT2
```

In the call $g((int*)nullptr)$, the constraints are not satisfied because no values can be deduced for the placeholders $C1$ and $auto$ from the expression $*t$ when t has type “pointer-to- int ”. – end example]

Change in 17.10.2 [temp.constr.decl] paragraph 2:

Constraints can also be associated with a declaration through the use of ~~template-introductions, constrained-parameters in a template-parameter-list, and constrained-type-specifiers in the parameter type list of a function template~~. Each of these forms introduces additional constraint-expressions that are used to constrain the declaration. A template's associated constraints are defined as a single constraint-expression derived from the introduced constraint-expressions using the following rules.

(2.1) – If there are no introduced constraint-expressions, the declaration is unconstrained.

(2.2) – If there is a single introduced constraint-expression, that is the associated constraint.

(2.3) – Otherwise, the associated constraints are formed as a logical AND expression (8.14) whose operands are in the following order:

~~(2.3.1) the constraint expression introduced by a template introduction (17.2), and~~

(2.3.2) – the constraint-expression introduced by each constrained-parameter (17.1) in the declaration's template-parameter-list, in order of appearance, and

(2.3.3) – the constraint-expression introduced by a requires-clause following a template-parameter-list (Clause 17), and

~~(2.3.4) the constraint expression introduced by each constrained-type specifier (10.1.7.4.2) in the type of a parameter declaration in a function declaration (11.3.5), in order of appearance, and~~

(2.3.5) – the constraint-expression of a trailing requires-clause (Clause 11) of a function declaration (11.3.5).

The formation of the associated constraints for a template declaration establishes the order in which the normalized constraints (defined below) will be compared for equivalence (to determine when one template redeclares another), and the order in which constraints are instantiated when checking for satisfaction (17.10.1). The constraint-expressions introduced by constrained-type-specifiers in a variable type or in the declared return type of a function are not included in the associated constraints of a template declaration. [Note: These constraints are checked during the instantiation of the declaration. – end note] A program containing two declarations whose associated constraints are functionally equivalent but not equivalent (14.6.6.1) is ill-formed, no diagnostic required. [Example:

```
template<typename T> concept bool C = true;

void f1(C);
template<C T> void f1(T);
C{T} void f1(T);
template<typename T> requires C<T> void f1(T);
template<typename T> void f1(T) requires C<T>;
```

All declarations of f1 declare the same function.

```
template<typename T> concept bool C1 = true;
template<typename T> concept bool C2 = sizeof(T) > 0;

template<C1 T> void f2(T) requires C2<T>; // #1
template<typename T> requires C1<T> && C2<T> void f2(T); // #2, redeclaration of #1
```

The associated constraints of #1 are C1<T> && C2<T>, and those of #2 are also C1<T> && C2<T>.

```
template<C1 T> requires C2<T> void f3();
template<C2 T> requires C1<T> void f3(); // error: constraints are functionally
// equivalent but not equivalent
```

The associated constraints of the first declaration are C1<T> && C2<T>, and those of the second are C2<T> && C1<T>. – end example]

Change in 17.10.4 [temp.constr.resolve] paragraph 1:

Concept resolution is the process of selecting a concept from a set of concept definitions referred to by a qualified-concept-name, or from a set of declarations including one or more concept definitions referred to by a simple-template-id or a qualified-id whose unqualified-id is a simple-template-id. Concept resolution is performed when such a name appears

(1.1) – as a constrained-type-specifier (10.1.7.4.2),

(1.2) – in a constrained-parameter (17.1),

~~(1.3) – in a template introduction (17.2), or~~

(1.4) – within a constraint-expression (17.10.2).

Within such a name, let *C* be the concept-name or template-name that refers to the set of concept definitions.

Change in 17.10.4 [temp.constr.resolve] paragraph 3:

The method for determining the concept argument list depends on the context in which *C* appears.

(3.1) – If *C* is part of a constrained-type-specifier or constrained-parameter, then

(3.1.1) – if *C* is a constrained-type-name, the concept argument list is comprised of a single wildcard, or

(3.1.2) – if *C* is the concept-name of a partial-concept-id, the concept argument list is comprised of a single wildcard followed by the template-arguments of that partial-concept-id.

~~(3.2) – If *C* is the concept name in a template introduction, the concept argument list is a sequence of wildcards of the same length as the introduction list of the template introduction.~~

(3.3) – If *C* appears as a template-name of a simple-template-id, the concept argument list is the sequence of template-arguments of that simple-template-id.

Change the example in 17.10.4 [temp.constr.resolve] paragraph 4 to replace the use of abbreviated functions and template introducers with function template declarations:

[...]

If any concept arguments do not match a corresponding template parameter, the concept *CC* is not a viable selection. The concept selected by concept resolution shall be the single viable selection in the set of concepts referred by *C*. [Example:

```
template<typename T> concept bool C1() { return true; } // #1
template<typename T, typename U> concept bool C1() { return true; } // #2
template<typename T> concept bool C2() { return true; }
template<int T> concept bool C2() { return true; }
template<typename... Ts> concept bool C3 = true;

template<C1 T> void f1(const C1*); // OK: C1 selects #1
template<C1<char> T> void f2(C1<char>T); // OK: C1<char> selects #2

template<C2<0> T> struct S1; // error: no matching concept for C2<0>,
// mismatched template arguments
template<C2 T> struct S2; // error: resolution of C2 is ambiguous,
// both concepts are viable

C3{...Ts} template<C3... Ts> void q1(); // OK: selects C3
C3{T} template<C3 T> void q2(); // OK: selects C3
```

– end example]

Acknowledgements

The author would like to thank Andrew Sutton for his continued dedication to progressing the adoption of Concepts into the C++ working paper.

References

- [Concepts] "C++ Extensions for concepts", ISO/IEC technical specification 19217:2015.
http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=64031
- [ConceptsWP] "Working Draft, C++ Extensions for concepts"
- [P0464R2] "Revisiting the meaning of foo(ConceptName, ConceptName)"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0464r2.html>
- [P0557R0] "Concepts: The Future of Generic Programming"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0557r0.pdf>
- [P0587R0] "Concepts TS revisited"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0587r0.pdf>
- [ExploringConcepts] "Exploring Concepts"
<http://lists.isocpp.org/ext/2017/02/2000.php>